Adversarial Attacks on MNIST Classifier

1. What This Is About

For this activity I trained a basic CNN on the MNIST digit dataset and then tried to break it using different adversarial attacks. The idea was to see how a model that performs really well on normal data can still be fooled by tiny, almost invisible changes to the input image. I also tried out a couple of simple defense methods at the end to see if they help.

2. Dataset

I used the MNIST dataset, which has handwritten digits from 0-9. It has 60,000 training images and 10,000 test images, each a small 28x28 grayscale image. Loaded it directly through torchvision and converted everything to tensors.

3. What I Did Before the Attacks

Before jumping into the attacks, I first set up and trained a normal CNN so I had a working baseline model.

- Building the model

Built a simple CNN with two convolution + pooling blocks and two fully connected layers at the end. Nothing fancy, just a standard image classifier.

-Training

Trained it for 5 epochs using Cross Entropy Loss and Adam optimizer. Loss kept dropping steadily each epoch, so training went fine.

-Baseline result

Result: got 98.89% accuracy on the clean test set. So the model is solid on normal, unattacked images, which made it a good candidate to test attacks on.

4. Adversarial Attacks I Tried

Once the model was trained, I ran a bunch of adversarial attacks on it to see how easily it breaks.

-FGSM (Fast Gradient Sign Method)

This one adds noise in just one step, using the sign of the loss gradient to push the image in the direction that confuses the model the most. It's the simplest and fastest attack.

Result: dropped model accuracy from 98.89% down to 29.48% at epsilon = 0.25. That's a huge drop from just one quick step of noise, which honestly says a lot about how fragile the model is.

-PGD (Projected Gradient Descent)

Basically a stronger version of FGSM. Instead of one big step, it takes several small gradient steps and keeps the noise within a set limit, making it a much tougher attack.

Result: successfully changed the model's predictions on test images, confirming it's a stronger attack than FGSM.

-BIM (Basic Iterative Method)

Very similar to PGD, applies FGSM repeatedly in small steps instead of one jump, refining the attack a bit more each round.

Result: also managed to flip predictions on the sample images, similar behavior to PGD.

-DeepFool

This one is a bit different, it tries to find the smallest possible push needed to just barely cross the model's decision boundary. So the noise it adds is usually very subtle.

Result: fooled the model too, and the perturbation was noticeably less obvious compared to FGSM.

-CW (Carlini-Wagner)

An optimization-based attack, it tries to minimize the noise added while still guaranteeing the model gets it wrong. Known to be one of the stronger, more precise attacks out there.

Result: also successfully fooled the model on the test batch, predictions changed just like the other attacks.

5. Defenses I Tried

After seeing how easily the model got fooled, I tried a few basic ways to make it a bit more resistant.

Adversarial Training

Retrained the model using FGSM-attacked images instead of clean ones, so it actually learns what adversarial noise looks like and gets better at ignoring it.

Result: model went through 3 extra epochs of training on adversarial images, expected to make it more robust to similar attacks going forward.

Feature Squeezing

Reduces the color depth of the image (like going from full pixel range down to fewer bits) before feeding it to the model, which can wipe out some of the tiny adversarial noise.

Result: visually the squeezed image looked almost identical to the original, so it's a cheap defense that doesn't mess up the actual digit.

Defensive Distillation

Uses a temperature-scaled softmax to smooth out the model's output probabilities, making it harder for gradient-based attacks to find a clear direction to push the image in.

Result: output probabilities became much softer and more spread out compared to normal, which is the expected effect of distillation.

6. My Takeaway

Basically the model was great on clean data but broke down really fast once any adversarial noise was introduced, even a simple one-step attack like FGSM tanked the accuracy massively. Stronger attacks like PGD and CW were harder to notice visually but still fooled the model. The defenses I tried (adversarial training, feature squeezing, distillation) all seem like reasonable, low-effort ways to add some robustness, but none of them are a complete fix. Overall this activity really showed that high accuracy alone doesn't mean a model is actually safe or reliable.